POSITION PAPER

Tell Your Coding Agent to Work as an Architect First

Let it declare intent, let it show the design, let both pass the gates — then, and only then, let it code

Stanislav Rumega

21 July 2026 · rev. 5.21 · working toward Preprint v1.1 — 26 July 2026

AI coding agents produce code faster than they can say what they are building. This paper proposes that the missing discipline is not better generation but staged, checkable artifacts: the agent first declares a structured *intent* (a System Intent Model), then derives a machine-readable *design* (a Formal Architecture Model), and only then writes code — with a small deterministic gate reviewing each representation before the next is produced. The argument is framed as feedback control: sensors may be probabilistic, but the control law may not, and the loop that improves the rules must itself be governed. The method is presented as an engineering hypothesis — coherent, buildable, and built — with its empirical validation left to a blind cold-test the paper describes.

1 The Missing Intermediate Representations

Modern coding agents are astonishing at producing code and almost incapable of saying what they are about to build. That gap — between the speed of generation and the absence of stated intent — is where the dangerous bugs now live.

Watch a coding agent work on anything non-trivial — a payment worker, a retry loop, a supervised actor tree — and a pattern emerges. It reads the issue, and within seconds it is writing code. The code is fluent, idiomatic, often correct in the happy path. But nowhere in the process does the agent state, in any checkable form, the answers to the questions that actually kill systems in production: *What happens when this crashes mid-operation? Is this command allowed to be lost silently? How many of these can exist at once? What enforces that this invariant holds when the thing enforcing it is the thing that failed? How is it stopped?* The agent may have implicit answers buried in the code it wrote. It has no explicit ones a machine — or a reviewer — can inspect.

This is not a complaint about any particular model; it is a property of the workflow. The pipeline today is `prompt → code`. Intermediate representations between requirements and code are, of course, an old idea — design docs, ADRs, RFCs on the informal side; TLA+, Alloy, UML, Design-by-Contract, and session types on the formal side. But these were all designed for a world in which *humans* write the code. The formal ones demand a rigor most teams never adopt; the informal ones are prose — unverifiable, unenforceable, written (if at all) for humans to skim, not for machines to check. What is missing is not the *idea* of an intermediate representation; it is an intermediate representation **shaped for an agent-first workflow** — one the agent itself can be required to produce, and a machine can deterministically check. The faster generation gets, the wider this gap grows. An agent that produces a thousand lines a minute with no stated intent is not an engineer; it is a very fast typist with opinions.

> **Thesis.** Before a coding agent writes code, it should have to declare — in a structured, machine-checkable form — what it believes the system is, how it can fail, and what it claims will hold. We call that artifact a **System Intent Model** (SIM), and a deterministic reviewer should validate it *before* any code exists to review. And it should have to do it twice: the approved intent is lowered into a concrete design — the **Formal Architecture Model** (FAM) — which a second deterministic reviewer validates in turn, checking the things that only become checkable once the design exists: completeness, consistency, traceability. Intent reviewed, then design reviewed, then the code itself — where mature static analyzers already exist and are adapted, not reinvented. And where the requirements themselves are too thin to declare honestly, the agent questions them — and, past the threshold, declines to build rather than guess.

The Same Thing in Plain Language

AI coding assistants start typing code immediately, without ever saying what they think they are building. So: before it writes any code, the assistant must fill in a short **questionnaire** — what parts the system has, how they talk to each other, what happens when something crashes, and what it is *not sure about*. A small, boring, reliable program — not another AI — checks those answers against a checklist of mistakes that real systems have made in production. If an answer is missing or fishy, the assistant must fix the *plan* and resubmit. Then it happens **again, one level down**: from the approved plan the assistant must draw the detailed **blueprint** — the exact tables of what each part does in every situation — and a second boring program checks the blueprint for holes: missing cases, states that can suspend unfinished work forever with no deadline, promises with no named place in the code that will keep them. Only when both checkers pass does anyone write code. And the code gets checked too — but here the industry already did the work: good automated code checkers exist for every ecosystem, so the pipeline simply *uses* them. The part nobody had built was the two checkers above it.

Why boring is the point: you do not want a second AI judging the first — it can be sweet-talked, just like the first one. The checker is a smoke detector, not a critic: simple rules, no opinions, same answer every time. *Is this message allowed to vanish silently? What stops a crash–restart loop?* If the plan does not say, that is a finding. And if *your requirements* do not say, the assistant must ask you — and if you will not answer the questions that actually matter, it must decline to guess. That refusal is not a tantrum; it is the job.

We say	We mean
SIM	the questionnaire the AI fills out before coding (a structured file)
gate / reviewer	the boring checker program — three, actually: one for the questionnaire, one for the blueprint, and off-the-shelf scanners for the code
heuristics H1–H14	the checklist for the questionnaire — “mistakes real systems have made”
checks F1–F6	the checklist for the blueprint — not stories from the past, but arithmetic: every case has a row, every promise has a place in the code, nothing appears that the questionnaire didn’t allow
blocking finding	a missing answer serious enough to stop the work until answered
FAM	the detailed blueprint drawn from the approved questionnaire — the *design*, a.k.a. the *architecture* (same thing); code must traceably implement it
decision memory	the project diary: every design choice and *why* — so you never re-argue it next month

If the rest of this paper ever gives you a headache, this box is the whole idea. The jargon exists for precision, not for reading.

2 What a System Intent Model Is

Definition. A System Intent Model is a structured declaration of a module's intended architecture, written by the agent that is about to implement it, covering: its *actors* and their responsibilities and state persistence; its *messages* with direction, delivery semantics, and acknowledgement contracts; its *guarantees* and the concrete mechanisms claimed to provide them; its *failure model* — what can go wrong and the stated response to each;

its *resource creation* and bounds; its *enforcement mechanisms* and the layer at which each invariant is enforced; its *identity and freshness assumptions*; and, explicitly, its *ambiguities* — the things the agent does not know.

Two properties make a SIM different from the design documents engineers already write, and both are essential.

It is structured, not prose. A SIM is JSON against a schema, with enumerated fields: a message's `delivery_semantics` is one of `at-most-once / at-least-once / exactly-once / unspecified`; a failure response has a `response_kind` of `detection / enforcement / unspecified`; an invariant's enforcement has a `layer` of `resource / platform / application / none`. This is what makes it *checkable*: a deterministic program can read it and ask precise questions, because the answers are in fields, not buried in adjectives.

A caution follows immediately, because it is the easiest way to misread the method: **schema conformance is necessary, but it is not semantic validity.** A SIM or FAM can be perfectly well-formed — every field present, every enum in range — while describing the wrong system, omitting a relevant failure mode, or faithfully preserving a mistaken requirement. The schema makes the claims explicit and addressable; it does not make them true. The gates, and the independently resolved evidence, are what decide whether a particular claim is admissible — so the method is not “ask the model to generate architecture JSON and validate it against a schema,” and should not be mistaken for it.

Absence is data. This is the discipline that makes the SIM honest. If there is no acknowledgement, the agent must write `"expects_ack": false` — not omit the field, and not write prose about how it will "handle reliability." If a failure has no response, the `response` is empty. If an invariant is enforced only by the same software it constrains, the layer is `application` or `none` — a statement that the enforcement is advisory, recorded as such. A negation ("not fenced", "no backoff") is recorded as the *absence* of a mechanism, never dressed up as its presence. In a SIM, what is *not* there is as visible as what is — a discipline closer to how a compiler treats a program than to how a design doc treats a system.

And there is a third, quieter feature: the `ambiguities` list. A coding agent that is forced to declare intent will sometimes not know the answer — is this subscriber reading a fresh value or a cached copy? what does this timeout actually do when it fires? The SIM gives it a place to say so, honestly, as a flagged ambiguity rather than a fabricated certainty. A model that can admit ignorance in a structured field is far safer than one that must guess to proceed.

3 The Right Frame: Intermediate Representations, Plural

It is tempting to read the SIM as a kind of design document that happens to be JSON — an artifact for a reviewer to look at. That undersells it. The more precise frame comes from an unexpected place: compilers.

A mature compiler does not translate source directly into machine code. It lowers the source through a series of *intermediate representations* (IRs), each one more explicit and more checkable than the last, and it runs verification passes on those IRs before any code is emitted. Note the plural — serious compilers have *several* IRs: LLVM lowers from LLVM IR through SelectionDAG to machine instructions; MLIR made “many dialects, progressive lowering” its entire design philosophy; each level has its own verifier, and the higher levels are semantic and sparse while the lower ones are structural and total. The IR is the load-bearing invention: it is what lets a compiler be *staged* — parse, then check, then optimize, then emit — with each stage trusting the last because the representation between them is precise. Nobody mistakes the IR for bureaucracy; it is the reason compilers are reliable. Our pipeline inherits the plural directly: the SIM is the top-level IR (semantic, tolerant of declared absence), the FAM is the second IR (structural, intolerant of it), the code is the emission — and each level has its own verification pass.

The SIM is exactly this, two levels up from code — with the FAM one level up, in between. Today's agents compile `Prompt → Code` in a single opaque step. The proposal here is to give that pipeline its missing IRs, one per level, as Figure 1 shows.

Figure 1. The generation pipeline with two intermediate representations and three deterministic gates. The SIM (intent level) is checked for the promises; the FAM (design level) for the arithmetic; the code (implementation level) for the defects that can be found only at the code level. The agent appears at the two IR stages, which it authors; everything to their right — the three gates — is deterministic and inspectable.

The SIM is the *first* intermediate representation — the intent-level one — between natural-language requirements and implementation, and the deterministic reviewer is the verification pass over it. The Formal Architecture Model is a *lowering* — and, by the standard of Section 3.1, itself a full IR, the design-level one: it has defined semantics, a machine-checkable form, its own verification pass (Gate 2), and a lowering beneath it (the code). This is not a terminology upgrade; it is the property that lets the FAM cross-examine the SIM (Section 6). The two IRs divide labor the way compiler IRs do: the SIM is semantic and sparse — absence and ambiguity are first-class citizens, honestly declared. The FAM is structural and total — absence is illegal there; a missing row is a finding, not a field. A note on names before we go further: throughout this paper, **“architecture” and “design” are the same thing** — the single intermediate level between the declared intent and the code, with a single set of artifacts. A companion rule, since the words invite confusion: **“requirements” name the prose the agent starts from; “intent” names what the agent declares in the SIM** — intent is requirements made explicit, structured, and checkable, not a synonym for them. The ladder is requirements → intent (SIM) → design (FAM) → code. We use “architecture” when the emphasis is on *structure* (actors, channels, supervision trees, ports) and “design” when the emphasis is on the *act and artifact of deciding* that structure; the FAM is both words at once, which is why its name can carry either. Nothing in this paper distinguishes them, and any reader who finds the two words pulling apart should report a bug in the text, not discover a distinction in the method. And the code is the final emission, traceable back through each stage. This framing does two useful things. First, it explains the architecture to any programmer in one sentence, using an analogy every programmer already trusts: *we are not inventing bureaucracy; we are moving software generation toward the same staged, verified pipeline that made compilers reliable.* Second, it makes the downstream pieces fall out naturally — the gate, the FAM, the compliance evidence report, and even the code are all just *consumers* of the IR. Get the IR right, and the rest of the pipeline is ordinary engineering. Get it wrong, and no amount of downstream rigor helps — which is precisely why the next two sections care so much about reviewing it, and about whether the agent can write it accurately.

There is a deeper reason the IR framing earns its keep: **a useful IR is the contract between stages.** In a compiler, each IR is the fixed point the passes on either side agree on — produced faithfully, preserved semantically, reasoned about directly. The same holds here, at both levels. Gate 1 reasons *about the SIM*; the FAM *refines* it without changing its meaning; Gate 2 reasons *about the FAM*; the code *implements* it; and traceability is the demonstration that the semantics were preserved across each stage. Once the IRs are understood as the contracts, the pipeline's stages stop being a sequence of ad-hoc artifacts and become a single, composable whole — which is exactly the property that made compiler IRs so powerful.

3.1 Properties of the IR

Promoting the SIM to IR status invites the questions compiler researchers ask of any IR. They deserve direct answers.

- **Is it lossless?** No — and deliberately so. An IR is an abstraction, not a transcription. The SIM records the failure-relevant semantics of the design, not every implementation detail. The right property is not losslessness but *adequacy*: does it capture everything the verification passes need to reason about? That is an empirical question, answered by the rubric and the cold-test, not assumed.
- **Is it canonical?** No. Many prompts can normalize to the same SIM, and the same intent admits multiple valid SIMs. Canonicity is not the goal; *fidelity* is — the SIM must describe the system the agent actually intends to build, which is precisely the "code match" property the validation measures.
- **Is it deterministic?** Split, by design. *Generation* of the SIM is stochastic (the agent proposes). *Review* of the SIM is deterministic (the same SIM always yields the same verdict). This is the central asymmetry of the whole architecture: a stochastic front end, a deterministic verifier.
- **What is the transformation invariant?** Not injectivity — *traceability*. Every FAM element cites the SIM element it refines; every FAM invariant names the code location that will enforce it. The invariant across stages is that meaning is preserved and *provably so*, by an audit trail, rather than by a one-to-one mapping. Enforcing that the code still honors the FAM across the *next fifty edits* is the declared-vs-actual drift problem — the pipeline's hardest open stage, addressed in Section 7.1.

Answering these plainly is not a concession; it is what separates an IR from a data format. A JSON blob becomes an intermediate representation when its properties — what it preserves, what it abstracts, and what the stages may assume about it — are stated and defended. The FAM answers the same questions at its own level; that is what makes it the second IR rather than a rendering of the first.

It is worth adding that this pipeline is an old idea whose tooling has only now caught up. In the extreme case the layering collapses entirely: an actor whose behavior is a complete Mealy table stored as data and walked by stable plumbing has its design and its code *coincident* — one artifact, two roles — and the LabHSM toolkit, an early actor-model framework for LabVIEW whose actors were defined by hierarchical state machines, said so on an animated banner twenty years ago: *“Design is code!”* The claim was right then; what was missing was a machine that could check the design before running it. That is the piece this paper adds.

4 Why Review Is Necessary — and Why It Must Come First

The case for reviewing the SIM before code rests on three observations, each drawn from real failures rather than theory. As will become clear, each observation applies again one level down — to the FAM before code — because the FAM is where the intent becomes concrete enough to check completely; we note the echo where it lands.

4.1 The Bugs That Matter Are Design Bugs, and They Are Invisible in Code Until They Fire

The incidents that take down production systems are rarely syntax errors or off-by-one mistakes. They are failures of intent: an acknowledgement that was never required, an instance count with no bound, an identity tied to a reusable label, an invariant enforced by the very party it was meant to constrain. We distilled a set of review heuristics from fifteen real systems — actor clusters, Kafka ingestion pipelines, consensus implementations, cellular architectures,

durable-execution runtimes, an industrial control system — and the pattern is consistent. A subset of these bugs is invisible even at the intent level and only becomes *checkable* at the design level: a missing (state, event) case is not a promise anyone failed to make — it is arithmetic that does not yet exist until the table is written. That is why the review runs twice: the SIM review catches broken and missing *claims*; the FAM review catches incomplete *constructions*. The Petabridge split-brain was a node-identity assumption (a reusable `hostname:port` mistaken for a unique incarnation) plus a stale local view of cluster membership; both were design decisions, both were invisible in any single diff, and both were catastrophic. A PagerDuty outage was an unbounded producer created per request against a shared broker — a resource-bound bug, present in the design before a line of code was written. These are not coding errors. They are *declared-intent* errors, and they can only be caught where intent is declared.

4.2 Reviewing Intent Is Cheaper and Earlier Than Reviewing Code

A code review of a thousand generated lines is slow, and it anchors the reviewer on what *is* rather than what *should be*. A SIM is a few hundred lines of structured declaration that can be reviewed deterministically in milliseconds — and, crucially, *before* the code exists to anchor anyone. Catching "this command is fire-and-forget with no acknowledgement" at the SIM stage is a one-line fix to a design decision. Catching it after the code is written means unravelling an implementation. The economics are the same argument the industry already accepts for shifting testing and security left — applied to intent, which is further left than either. The FAM sits one rung closer to code but still far above it: a complete Mealy table for a payment worker is a page of rows, checked in milliseconds, where the equivalent assurance from code means reconstructing the table from control flow — the extraction problem we deliberately avoid by declaring first. Each level down costs more to check than the level above it and less than the level below; that gradient, not dogma, is why the gates are ordered the way they are.

4.3 An Agent That Must Declare Intent Catches Its Own Mistakes

There is a subtler benefit, and it may be the largest. The act of writing the SIM forces the agent to confront the design before committing to it. An agent that has to write down `"delivery_semantics": "at-most-once", "expects_ack": false` next to a field it has named `ChargePayment` has, in that moment, a chance to notice the problem itself — before any reviewer runs. Declaring intent is not only an input to review; it is a discipline that improves the design being declared. In our closed-loop reference run, the agent's second and third SIM proposals were not merely compliant — they were *better designs*: the fire-and-forget alarm gained an acknowledgement, the in-memory dedup moved to a resource-layer constraint, the unbounded instance pool gained a bound. The gate's findings drove the improvement, but the declaration is what made the improvement possible. The same discipline operates at the FAM level, in a sharper form: an agent that has to write the *row* for “ShutdownWorker arrives while Charging” cannot leave the case unconsidered — the table format itself forces the question. Half of what Gate 2 catches is not disagreement with the agent's design but the design's unexamined corners, made examinable by the requirement to fill them in. To be clear about the epistemic status: this is a single anecdote — an *existence proof* that the loop can run and can improve a design, not evidence that it does so reliably. Reliability is exactly what the blind A/B cold-test (Section 6) is designed to measure, and nothing in this paper should be read as pre-empting that experiment.

4.4 Loop Zero: Question the Requirements — and the Right to Refuse the Build

There is a loop before the first gate, and it is the only one whose converging partner is not a machine. An agent told to work as an architect will sometimes — often — discover that the prose requirements it received are not enough to fill the questionnaire honestly: no word on what happens when the gateway is down, nothing about whether the

command may be duplicated, silence on who may stop the system. Today's agents paper over such gaps with plausible defaults and type on. That is precisely the behavior this pipeline exists to stop — so the discovery itself must be part of the method.

Loop Zero works like this. The agent drafts what it can, and for each gap it cannot honestly fill it records a *blocking ambiguity* — the SIM schema has a place for exactly this, severity and all. Blocking ambiguities do not route to another guess; they route *back to the human*, as questions posed in the SIM's own vocabulary: not “tell me more about the system,” but “when the payment gateway is unreachable mid-charge, should the worker retry with a bounded backoff, park the order, or escalate?” Each question is a schema slot, each answer edits fields rather than prose, and the exchange is the decision memory's first and often richest feed — for many systems, this interrogation is the first time the requirements were ever written down in checkable form. Two disciplines keep the loop survivable: questions are *batched*, and each carries the agent's default proposal (“if unanswered, I will assume at-least-once with an idempotent receiver”) — defaults make silence answerable, the record makes it honest.

None of this abolishes the ambiguities list — it *routes* it, by severity, and the routing is the design. **Blocking** ambiguities interrogate, and block the build if unanswered. **High** ones are asked, but the agent may proceed on an explicit default — on record, as always. **Medium and low** ones simply stay in the document: honestly named, never silently guessed. A SIM with a populated ambiguities section is healthier than one pretending completeness, because the list is where the honesty lives. The discipline is asymmetric on purpose: the agent must *surface* everything — exhaustively — but may only *refuse* on the things where the failure modes live. Surface everything, refuse selectively: a pipeline that interrogates the user about trivia is uninstalled within a week, exactly like a gate that cries wolf. And one refinement worth stating: if the user answers a blocking question with “I don't care, just pick,” that is a *resolution*, not a refusal — the human delegated, the decision memory records the delegation (“user accepted agent default X”), and the build proceeds with the default marked user-sanctioned. Even indifference becomes a decision, on the record.

One more refinement is forced by reality: **the user is often not qualified to answer, and that has to be survivable.** A bakery owner who wants a loyalty app cannot say what delivery semantics her payments should have — and should never be asked in those words. The first move is therefore *translation, not omission*: the agent poses each blocking question in consequences the user owns — “if a customer pays and the connection drops, is it worse to charge them twice or to lose the order?” — and maps the answer back to the schema slot. The SIM vocabulary is for the gate; the user speaks outcomes, and a person who cannot define idempotency can still say “worse to double-charge.”

And where even a translated question cannot be answered, the second move: **the agent is allowed — expected — to know better than the user.** This is the part of the design that inverts the usual flow of authority, and it should be stated plainly. The user owns the *what* and the *why*: what the system does, what it may never do, what a failure costs in their world. The agent routinely knows the *how* better than any individual user can: which delivery semantics make double-charging impossible, which dependency versions carry known vulnerabilities, which logging satisfies an auditor — and what the law now requires of software in its class. Regulatory constraints like the EU Cyber Resilience Act are the sharpest case: the user has never heard of an SBOM, yet the agent retrieves the CRA's versioned requirements from authoritative sources, maps the declared scope onto machine-checkable controls at the SIM, the FAM, and the code, and emits a *compliance evidence report* and the bill of materials as byproducts — flagging the questions it cannot settle (applicability, classification, conformity assessment) for qualified human review rather than certifying anything itself. The CRA's main obligations apply from 11 December 2027, and an SBOM is only one provision among many; the agent assembles the evidence, it does not sign the assessment. These choices go on record as *posture defaults* (“strictest reasonable option assumed, user did not override”), and the gate may *require* the strict default where the SIM declares a compliance scope. The asymmetry is deliberate and

defensible: what the user must answer is what only they can know — the system's purpose and its irreversibles; what the agent must supply is what only it can know — the state of the art and current regulatory requirements retrieved from versioned authoritative sources. A harness that waits for users to become experts protects nobody; a harness that applies expertise and records the application is precisely the point of having one.

And if the human will not cooperate past the blocking threshold — that is, cannot or will not answer even the purpose-level questions, the ones about what the system should *do* and what it must never do — then the agent must be allowed — required — to **decline the build**. This is not petulance; it is the same professional standard every mature discipline already enforces: a structural engineer does not sign off on vague loads, a surgeon does not operate on “somewhere around here.” An agent that proceeds on unresolvable ambiguity is not being helpful, it is being negligent — it will produce a SIM that passes every gate while describing a system the user never asked for, a compliance certificate for a fantasy. Note who does the refusing: not the agent's judgment, but the pipeline's — a blocking ambiguity means Gate 1 cannot pass, which means no FAM, which means no code. The refusal is reproducible, auditable, and immune to sweet-talking, which is exactly why it lives in a deterministic gate rather than in the agent's discretion. And it is a *report*, not a sulk: the deliverable of a refused build is the documented hole — the blocking ambiguities, the questions, the defaults proposed, the options considered — often the most valuable artifact the project has produced so far. One legitimate exit remains: *shrink the scope* to what is genuinely specified (“the logging component's requirements are complete; the payment path stays unbuilt”). Building the well-specified subset is cooperation; widening the guessed scope to look complete is not. Trivial gaps never trigger any of this — that is what defaults-with-record are for. Refusal is reserved for ambiguities where the failure modes live: delivery semantics of money-moving commands, shutdown authority, identity assumptions. The right to refuse is what makes the agent an architect rather than a typist. Architects can be wrong on the record; typists just retype.

5 Why the Review Must Be Deterministic — Not Another Model

If the SIM — and the FAM derived from it — is to be reviewed, the obvious question is: reviewed by what? The tempting answer — another LLM acting as a judge — is precisely the wrong one, and it is worth being clear about why.

A stochastic judge cannot be the fixed rulebook. The entire point of the review is to be a hard, reproducible constraint on a fast, probabilistic generator. An LLM reviewer is itself probabilistic: the same SIM may pass today and fail tomorrow, and the agent being gated can learn to write SIMs that *please the judge* rather than SIMs that are *sound*. This is the LLM-judges-LLM trap, and it recreates the original problem one level up. Worse, an LLM judge's reasoning is not auditable — you cannot open it and read the exact rule that fired. The review we need must be the thing the generator is *not*: slow, small, deterministic, and inspectable.

The principle, stated plainly: the stochastic mind proposes; the deterministic record disposes. The reviewer is a fixed set of checks — H1 through H14 plus the property-based H5e–H5g — sixteen deterministic checks implemented as ordinary, auditable code, plus one question (H11, arbitrary invariant conflict) routed to a human-review lane because it needs a mitigation-operation model the schema does not yet carry; each traces to a real incident or a named failure class. For its deterministic checks, the same SIM always yields the same machine report — the human judgment that may follow is, deliberately, not deterministic. (Where a learned analyzer contributes evidence, the guarantee narrows: see Section 7.5 — the verdict is deterministic over a *frozen* evidence bundle, not over a fresh observation.) A design partner, or a regulator, can read the rule that produced a finding. That determinism is not a limitation; it is the entire value. It is also what makes the review *composable* into a gate: a deterministic check can block a build; a probabilistic opinion cannot.

Stated generally, the rule is not “LLMs should not review.” It is: **a probabilistic component may discover or observe, but it must not be the ungoverned final authority on whether a gate passes.** Probabilistic decision-making is legitimate elsewhere — Bayesian policies, stochastic control, risk-sensitive optimization all decide — so the claim is narrower and more defensible than a blanket “none may decide.” A sensor may well emit *probability of exploitable injection: 0.93*; what the gate then applies is an explicit policy — block when the probability exceeds a threshold, in a declared enforcement path, from an approved analyzer version — over evidence that is *frozen and recorded*, so the verdict is reproducible even though the measurement was not. Sensors may be probabilistic; gate policy must be explicit and reproducible over frozen evidence. Discovery and observation tolerate variance — a scout that misses one failure class this week finds it next week, a sensor with a false positive is caught by the human it escalates to. Decision does not tolerate variance, because the decision is the contract. Nor does the rule claim deterministic checks are always *better*: for a fuzzy judgment — “this architecture feels overly coupled” — a model may genuinely outperform a hand-written rule, and the right division is that the model *raises the concern* while the gate blocks only on an explicit deterministic policy that says why. Concerns are cheap; blocks must be earned.

It is worth naming what this architecture resembles, because it is older than AI: a governed feedback architecture, in the family of feedback control. The vocabulary of Section 8.1 already says so — scout, sensor, gate — and it maps onto the oldest pattern in control engineering: something expands the controller’s knowledge, something measures the system, something makes the control decision. The paper’s claim is correspondingly not “AI code should be checked” but “engineering systems have always separated discovery, measurement, and control — AI pipelines should do the same.” The mapping is worth stating exactly, because it is easy to misread. **The coding agent is the plant** — the stochastic, adaptive system whose behavior is being regulated: it emits SIMs, FAMs, and code in response to structured corrective signals. **The surrounding harness is the controller**: it observes those artifacts through its sensors (the schemas, the analyzers, the extractors), compares them against a reference, and returns findings or permits the next stage. The reference is not the vague setpoint “good software” but a progressively fixed contract — prose requirements, then the accepted SIM, then the accepted FAM, then the code-level policies — so the controller never optimizes an unrestricted notion of quality; it regulates the plant toward declared commitments. And the control signal is real, not advisory: blocking findings change what the agent may do next — revise the SIM, answer this question, derive the FAM, proceed, stop. The agent is an unusual plant, to be sure: it reads the controller’s findings semantically, redesigns itself in response, generates its own intermediate descriptions, and could in principle try to persuade the controller. That does not break the analogy; it explains why independent enforcement matters so much. A plant intelligent enough to produce artifacts that merely *appear* compliant is exactly why the controller must operate over explicit representations and remain outside the plant’s authority.

The scout belongs to a different loop, on a slower timescale, and conflating the two is the easiest misreading of all. The *operational loop* is fast: gate findings regulate the coding agent within a single build — output, measurement, structured corrective feedback, revised output, until each gate passes. The *adaptation loop* is slow: accumulated failures and experience, scouted and admitted under human review, revise the controller’s own policy — so that future builds are regulated differently. The first loop never changes the rules; the second loop never touches a single build. Keeping them apart is what lets the system learn without letting the plant rewrite its own controller.

And the determinism itself has three distinct layers, worth telling apart because they fail differently. *Deterministic execution*: the gate gives the same decision for the same input, every time. *Deterministic representation*: the SIM and FAM constrain semantics into machine-checkable forms, so there is something stable to execute over. And — least obvious, and the subject of Section 8 — *governed adaptation*: the reviewer itself changes only through an explicit admission process — scout proposes, human admits, decision event records, deterministic implementation lands. This third one is deliberately *not* called deterministic: a human decides whether to admit a heuristic, and two humans, or the same human on two days, may decide differently. What is guaranteed is not the outcome but the

process — constrained, recorded, versioned, externally authorized. Call the three, precisely: *reproducible execution*, *checkable representation*, *governed adaptation* — the rules evolve under governance, even though the evolution itself is a human judgment.

One refinement keeps the philosophy honest, and it is the deepest point in this section: **the decision procedure is deterministic; the world model is not.** Nobody — human, model, or program — ever has complete information. Choosing a retry budget of five, a mailbox depth, a timeout — these are estimates under uncertainty, expected-value judgments over futures nobody can enumerate. The gate knows this and claims nothing about it. It does not ask whether five is the *right* budget; it asks whether the architecture contains a mechanism enforcing the budget that was declared. The uncertainty lives upstream, in the engineering judgment; the enforcement lives downstream, over the declared commitment. This is simply how engineering already works everywhere else: nobody knows the true failure probability of a new wing design — it is estimated — and then a deterministic certification rule is applied to the declared design. The certificate does not say "this aircraft is objectively safe"; it says "given our current knowledge, it satisfies every required criterion." Those are different statements, and the whole strength of the method depends on keeping them apart. So the sharper formulation of the paper's slogan is this: **the uncertain world is explored probabilistically; the commitments made in response are enforced reproducibly.** Estimate probabilistically; commit explicitly; enforce reproducibly. The gate does not eliminate uncertainty — it records the decision made under uncertainty ("chosen retry budget: five; rationale: expected outage cost") and thereafter verifies, mechanically, that the implementation honors what was declared. The contrast was never really *stochastic versus deterministic*; it is *beliefs versus commitments* — and each belongs to a different stage, with a different tool.

6 The Feasibility Question: Can an Agent Even Write a Good SIM?

This is the honest crux, and we will not pretend it is settled. The entire pipeline stands or falls on a single assumption: **that the SIM captures the agent's actual intent accurately.** If the agent writes a tidy, internally-consistent SIM that describes a system other than the one it is about to build, then the deterministic review, the formal model, and the code can all be perfectly consistent with one another — while implementing the wrong thing. This is the risk an independent reviewer named when shown the architecture, and it is the correct diagnosis.

Three things make the assumption *plausible* rather than naive — and a fourth belongs to the FAM: the lowering is a *second, independent sampling* of the same intent. An agent whose SIM quietly described the wrong system would have to carry the same misreading, consistently, through a far more constrained artifact — typed messages, complete tables, named code locations — and then past a gate that checks the FAM against the SIM mechanically. Drift between SIM and FAM is detectable in a way drift between SIM and code is not, precisely because both artifacts are structured. The FAM is thus not only a design step; it is the intent's first cross-examination.

Declaration is easier than extraction. The hard version of this problem — reading existing prose or code and reconstructing its intent — is genuinely difficult and error-prone. But that is not what we are asking. We are asking the agent to *declare* its intent before implementing, in a structured form it is entirely capable of emitting. Producing JSON to a schema is squarely within a coding agent's competence; the discipline is in requiring it, and in refusing to proceed without it. The SIM inverts the extraction problem: you do not recover intent from artifacts, you require intent as a precondition for artifacts.

The schema is honest about ignorance. The anti-fabrication disciplines — absence is data, negation is absence, ambiguities are allowed — mean the agent is never forced to invent a mechanism to proceed. When it genuinely does not know, it says so in `ambiguities[]`, and the gate surfaces that as a question for a human rather than letting

it harden into a false claim. A declaration format that has a first-class place for "I don't know" is far more likely to be accurate than one that must always produce an answer.

It is measurable, and we intend to measure it. The assumption is not to be taken on faith; it is to be tested. The validation protocol is a blind A/B comparison: the same agent solves the same issues with and without the gate, and the SIM is scored on a rubric whose most diagnostic dimension is *code match* — does the declared SIM describe the code the agent actually wrote? If SIMs are routinely tidy but describe a different system, the assumption is falsified and the declaration contract must be fixed before anything downstream is trusted. If they track the code, the gate is reviewing the real design. This is the experiment that decides whether the whole approach is sound — and it is designed to be able to fail.

7 The Method: The Loop That Enforces It

Assembled, the method is **three closed loops with three deterministic gates** — one per level of commitment. The agent declares intent; Gate 1 checks the promises; the loop repeats until they hold. The approved intent is lowered into a design; Gate 2 checks the arithmetic; the loop repeats until it is complete and consistent. The approved design is emitted as code; Gate 3 — the static analyzers every ecosystem already has, adapted and routed — checks the emission; the loop repeats until it is clean. Three times the agent proposes, three times a deterministic pass disposes, and only the third proposal is the one anyone wanted in the first place.

```
prose spec
  → LOOP 0: blocking ambiguities → questions to the human
        ↑___ answers edit SIM fields (defaults on record) ___↓
        (no cooperation past the threshold → decline, deliver the evidence)
  → agent DECLARES a SIM
  → GATE 1 reviews it (H1–H14, intent)
        ↑________ blocking findings fed back ________↓
  → iterate until the SIM passes
  → compliance evidence report + SBOM (byproducts of the passing review)
  → agent DERIVES the FAM — the design, machine-readable
  → GATE 2 reviews it (F1–F6, design)
        ↑________ blocking findings fed back ________↓
  → iterate until the FAM passes
  → code, traceably implementing the FAM
  → GATE 3 reviews the code (existing SAST tools, adapted)
        ↑________ blocking findings fed back ________↓
  → iterate until the code passes
  → every decision logged to the project's decision memory
```

Each stage exists for a reason, and each is only trustworthy because the one before it was gated.

Gate 1 runs fourteen heuristics — failure-mode completeness, delivery coherence, bounded recovery, no unbounded waits, identity and freshness integrity, verification strength, independent enforcement of invariants, bounded resource creation, operation reversibility, invariant conflicts, conditional guarantees, and corrigibility (can the system be stopped on command: enforced, inherited, and not through a single point of failure) — plus three property-based additions covering actors whose behavior cannot be enumerated, and a supply-chain check on every external dependency. **Gate 2** runs six of its own over the FAM (F1–F6, Section 7.4): table completeness, block–property match, lowering integrity, port coherence, guard-as-code, code-mirror readiness. **Gate 3** adapts existing

analyzers for the code (Section 7.5). The full question lists are in the Appendix; the implementations, with severities and applicability conditions, are a few hundred lines of ordinary, auditable code (a small worked example — one SIM, its derived FAM, and the three gate reports — accompanies this paper; the full schemas and the curated knowledgebase are released separately, once the validation described in Section 10 has run) each. Findings are severity-ordered, each with the evidence and the question to ask. A *blocking* finding stops its loop; a strength is confirmed as sound engineering to keep.

The iteration is the point. When a gate blocks — at any of the three levels — the findings go back to the agent — not as a human's red ink, but as the precise, structured reasons the declared design is unsound. The agent revises the *design* (not hides the gap) and resubmits. In our reference run, the first SIM was blocked (fire-and-forget payment and alarm commands, no failure model, a guarantee with no mechanism), the second was blocked again (an in-memory "enforcement" that was really advisory, an unbounded instance pool), and the third passed — with a resource-layer idempotency fence, a bounded pool, a bounded restart policy, and a declared shutdown path. Three rounds, each one a measurably better design.

The FAM and the code come last, and only then. Once the SIM passes, the agent derives the design — the Formal Architecture Model, i.e. the architecture, written down: typed messages, explicit state machines with per-state transitions, named invariants mapped to their enforcement layers and code locations — and Gate 2 reviews that design with its own loop (Section 7.4) before any code is written. Only when both gates have passed does the agent write code that mirrors the FAM through explicit, machine-checkable mappings — and Gate 3 then reviews the code itself (Section 7.5). The code is no longer the first artifact; it is the last, and it is traceable back through the FAM to a SIM that a deterministic gate accepted. This is the agent-native realization of an old ambition — the unambiguous specification that precedes the program — made practical because the specification is now something the agent itself must produce and a machine can check.

And every decision is remembered. Each check, block, and override — especially the human overrides with their stated rationale — is appended to the project's decision memory, an append-only log that outlives any single session. (One could call it the project's “constitution”; we will resist that temptation.) Its one non-negotiable clause is that the agent may not amend it unilaterally; amendments are human decisions, recorded. This is what turns a static gate into a learning system: the heuristics themselves were each distilled from a real incident, and the memory is where the next ones come from.

One honest question remains, and it is the one a skeptic asks immediately: *why do these loops ever end?* It deserves a precise answer, because the obvious one is wrong. A deterministic gate does **not** guarantee convergence — it guarantees *reproducibility* (the same gate version, inputs, and evidence yield the same verdict), which is a different property. The space of possible SIMs and programs is effectively unbounded; nothing stops an agent from fixing H3 while reintroducing H4, alternating between two failing designs, or surfacing previously dormant checks forever. “The blocking set is finite” does not imply “the number of iterations is finite.” So zero blocking findings is the *success* condition, not the *termination* condition — and the two must not be conflated.

The actual guarantee comes from a separate **bounded-remediation policy**, which lives in the controller, not in the heuristics. Each loop has terminal outcomes beyond PASS — human override, escalation on no-progress, escalation on detected cycle, abort on budget exhaustion — and a declared policy drives toward one of them: run the pinned gate version; if blocking is zero, pass; otherwise issue a finite set of proof obligations; permit a bounded number of resubmissions; compare each report with the last; if blocking findings decrease, continue; if the same artifact-and-report pair repeats, declare a cycle; if findings persist without progress, escalate to a human; record every waiver, override, or abandonment in the decision memory. The gate supplies what a probabilistic reviewer cannot — stable verdicts, explicit obligations, and reports comparable across iterations, which make progress and cycling *machine-*

detectable — but it is the loop policy that forces every run to terminate in pass, waiver, escalation, or abort. This is also the honest reading of how this very paper was reviewed: an LLM reviewer kept moving the acceptance boundary, and the loop only closed when a fixed criterion was declared from outside. A deterministic gate stabilizes the boundary; the bounded-remediation policy is what guarantees the loop respects it.

7.1 Keeping the Declaration True: The Drift Problem

The sharpest objection to everything so far is not about the first build — it is about the *next fifty edits*. A SIM approved on Monday is a claim about a system that, by Friday, has absorbed a hotfix, a new message, and an "obviously harmless" timeout change. If the declaration silently stops describing the code, the gate becomes a ceremony: impressive on day one, decorative by week two. This is the drift problem, and a position paper should say plainly how the architecture intends to meet it.

The design answer is that **drift is checkable because extraction is symmetric to declaration**. The same pipeline that lets the agent *declare* a SIM before code can be run in reverse over existing code — re-extracting a candidate SIM (or FAM) of what the code *actually does*. Drift detection is then not a semantic guessing game but a **diff between two structured documents**: the approved SIM and the re-extracted one. New message without a declared delivery semantics? A failure response quietly emptied? An unbounded pool where the SIM declared a bound? Each of those is a diff line, and the gate can run on the diff exactly as it runs on a fresh SIM — blocking, with a question, when the drift crosses a rule.

And drift is not always sin. Much of it is the system *learning*: the hotfix that adds a bound is a design improvement discovered in production. So drift is routed through the same discipline as everything else: the human reviews the diff, and if the new reality is accepted, the SIM is amended *with a recorded rationale* — a decision event in the memory, which is precisely the feed that grows the knowledgebase (Section 8). Drift the human rejects is a bug to fix; drift the human accepts is a lesson to keep. Either way the declaration, the code, and the record stay mutually honest.

Honesty about status: the extractor exists and has been run against real frameworks (Section 8.2's industrial actor framework was machine-extracted), but the re-extract–diff–gate loop is *prototype-stage*, not yet a daily tool. The drift check is also what makes the FAM earn its keep: FAM elements name the code locations that implement them, which is what gives the diff somewhere to bite. We state the design here because it follows from the architecture — not because we claim it is finished. If the cold-test says the SIM is accurate at birth, keeping it accurate is the experiment after that.

7.2 The FAM Without UML: One Skeleton, Four Evidence Shapes

A fair skepticism meets any proposal for a "formal architecture model" in 2026: *UML is dead, and good riddance — what makes your blueprint notation any different?* UML died of three causes: too many diagram types, no executable semantics, and tooling that could not keep picture and code in sync. The FAM avoids all three by not being a notation at all. It is a fixed skeleton with a small set of *evidence shapes*, and which shape an actor uses is determined by the same declared properties the gate already reads — never by a family label.

The skeleton is always the same five parts: **ports** (each with domain, dimensions, rate, bounds), **state** (its dimensions and persistence), **behavior** (one block, below), **guard** (the deterministic envelope around the behavior, as code with a named location), and **traceability** (which SIM elements this refines; which code locations implement each part). The behavior block comes in exactly four shapes:

Block	When (property, not label)	What the agent writes

T — Table	enumerable + reproducible (FSM, HFSM, behavior tree)	The full Mealy table, (state, event) → (actions, next state), every row including unexpected-event rows; hierarchies resolved to their leaf-level table. This is the case where the table *is* the code's behavior-data — the FAM and implementation coincide, and the gate can compute over the rows directly.
E — Envelope	not enumerable or not reproducible (learned policy, stochastic, genetic/swarm)	What cannot be listed must be enclosed: the valid input domain (stated symbolically), the guardrail predicates every output must satisfy (each with its code location), the deterministic fallback, and a few sampled *witness* examples for human review — spotlights, not the map.
D — Dynamics	continuous/hybrid ports (control loops, signal chains)	The update law stated symbolically (or as a named code location), per-port range, saturation and rate, and the invariants over them (anti-windup, slew limits). May combine with T or E.
G — Generative	code-generating machinery (parsers, table compilers)	The specification the generator consumes, the determinism guarantee (same spec → same artifact), and the validation step that checks the generated artifact.

7.3 Yes, This Is the Design — and No, There Are No Diagrams

At this point a certain kind of reader — let us call him, charitably, *the experienced architect* — shifts in his chair. He has been patiently translating our vocabulary into his own: the SIM is “requirements,” the gate is “review,” the FAM is “design” (or, in his other dialect, “the architecture” — same level, same artifacts, as Section 3 established). And now the translation produces an alarm: if the FAM is the design, *where are the diagrams?* No class hierarchy lovingly arranged, no component boxes with little gears, no sequence arrows choreographed across a swim-lane. Just tables, envelopes, port contracts, and traceability citations. To him, this does not look like design. It looks like design *skipped* — a pipeline that lurches from requirements straight into implementation, the way juniors do.

He deserves a real answer, because his mapping is correct. The FAM *is* the design — it occupies exactly the slot UML diagrams occupied in the process he grew up with: the moment where structure is decided and recorded before implementation begins, the artifact a newcomer is handed to understand the system, the thing review meetings are about. What has changed is not the slot but the *shape of the artifact that fills it* — and the change is forced by two facts about who now writes and reads it.

The primary *executable* consumer is no longer a person. A UML diagram was a human-to-human compression format: a senior engineer could gesture at boxes in a meeting, and the semantics lived in the viewers' heads. That is precisely why it rotted — nothing in the picture could disagree with the code, so nothing stopped it from doing so. The design artifacts in this pipeline are read first by machines: the gate checks them, the drift differ compares them against the code, the compliance evidence report is generated from them. A design whose reader is a machine must be *data* — structured, typed, absence-explicit. A picture cannot be diffed; a table row can. The experienced architect is right that something is lost when the whiteboard closes; what is gained is that the design can now be *false* in a machine-detectable way, which is the property that makes artifacts trustworthy at all.

The primary author is no longer reliable. A human designer draws what he intends and the drawing is the commitment; a coding agent produces whatever satisfies its local objective, and the only design worth having from it is one a deterministic checker can hold it to. A design artifact whose writer is stochastic must be *checkable* — every claim in it must be the kind of claim that can be verified mechanically, against a schema today and against the code tomorrow. Tables and envelopes are; swim-lanes are not. This is the deeper reason the FAM looks the way it does: it is design written in the only register that survives contact with an author who is fluent, fast, and not accountable.

None of this confiscates the diagrams. People genuinely need the at-a-glance overview — the supervision tree, the message choreography, the statechart — and they can have it, on one condition: **the picture is a view, not the source.** Every diagram in this pipeline is *generated* from the FAM the way API documentation is generated from code: always current, never authoritative, disposable without loss. The moment a diagram becomes editable it becomes a second source of truth, and two sources of truth are zero sources of truth — that, not aesthetics, is what actually killed UML tooling. The experienced architect keeps his wall chart; he just stops being able to "fix the design" by dragging a box. He will find, after the initial sting, that fixing the table and regenerating the picture takes less time than the argument the picture used to start.

So: yes, the FAM is the design. It is design after the audience changed — written for the reader that can actually check it, with diagrams regenerated on demand for the readers who cannot. The discipline was never the pictures; the discipline was deciding, explicitly, before building. That discipline is still here. It just types now.

Two properties of this design matter more than the blocks themselves. First, **the blocks are evidence shapes, not notations**: T is evidence by enumeration, E by enclosure, D by bounds, G by determinism-of-generation. A behavior paradigm nobody has seen yet — neuro-symbolic policies, diffusion planners — needs no new block; it lands in E, because E is defined by a property of the behavior, not by a school of thought. Second, **the gate knows which block to expect**: it reads the actor's declared properties from the SIM and checks that the FAM carries the matching shape — so the agent cannot dress a learned policy in a table, or hide a missing envelope behind a notation that cannot express one. Where pictures are wanted, they are *generated views* of these blocks (a statechart rendering of a table, a message-sequence rendering of the ports); where proofs are wanted, the blocks lower to verification targets (a table to a TLA+ next-state relation). The source of truth stays textual, diff-able, and checkable — which is precisely the trio of properties whose absence killed UML.

7.3.1 "But My Agent Already Plans": The Plan Is the Degenerate FAM

A second reader objection arrives from the opposite direction — not from the UML veteran but from the heavy agent user: *my coding agent already produces a plan before it writes code.* It does. Plan modes, todo lists, step-by-step implementation outlines — every serious agentic tool now emits some form of "first I will add the schema, then the endpoint, then wire the UI" before or while editing. Is that not the design level? Does it not deserve a gate of its own?

Look at the artifact's actual form. It is **prose** — a Markdown numbered list, sometimes flattened into checkboxes, occasionally with file names sketched in. Three properties follow. It is *unstructured*: no schema, no types, no totality criterion — nothing that would let a program say "step 4 is missing" mechanically. It is *unchecked*: no reviewer reads it, no gate blocks on it, and there is no completeness arithmetic it could be checked against — what would "the steps look right" even verify? And it is *ephemeral*: written for the session, discarded with it; nothing forces the code to match it, and nothing notices when it stops matching. The plan is the agent thinking out loud. It is scaffolding, not an intermediate representation.

What it is *trying* to be, however, is unmistakable: the plan says *what I will build and in what order* — which is precisely the question the FAM answers. The industry has independently discovered the need for a design-level IR between intent and code, and filled the slot with the weakest possible artifact: one that cannot be validated, cannot be totaled, and cannot be enforced. So the plan is neither a replacement for the FAM nor a third IR between design and code. A level earns a gate by admitting its own verification standard — a mechanical totality test under which absence is illegal. The SIM has one (the promises must all have mechanisms); the FAM has one (every state-event pair must have a row); code has one (it must compile and scan clean). The plan has none — its failures decompose without remainder into design-level failures (unrefined actors, missing rows) and code-level failures (the diffs don't

match), leaving no residue that is plan-specific. There is nothing for a Gate 2.5 to check that Gates 2 and 3 do not already check better.

The one genuinely characteristic plan failure — **plan–code drift**, the agent's stated steps diverging from its actual edits mid-session — is not a counter-example but the proof: it is exactly the drift problem of Section 7.1, and the answer is the same. Replace the unscheckable plan with a checkable design, and drift becomes mechanically detectable (re-extract, diff, gate) instead of invisible. The plan, in short, is the degenerate FAM: the design level attempted in prose, by agents that were never given a design language. Give them the language.

7.4 The Second Gate: Checking the Arithmetic

Everything through Section 7.3 solves the problem of getting the design *written*. It does not solve the problem of getting it *checked* — and a pipeline that gates intent but takes the blueprint on faith has merely moved the trust bottleneck one stage down. The first version of this pipeline had exactly that hole: the gate looped at the SIM level, and the FAM was derived afterward, unchecked. That was a mistake, and it is worth being public about its shape because the shape is instructive: the checks that *can* be run at the two levels are different in kind.

The SIM gate checks **promises**: is there a response to every applicable failure mode, a mechanism for every guarantee, a bound on every resource? These are questions about what the design *claims*. The FAM gate checks **arithmetic**: does the Mealy table actually have a row for every (state, event) pair? Does every waiting state actually carry a timeout? Do the blueprint's ports match the interface the SIM declared? Does every guardrail name the code location that will enforce it? These are not questions to ask an author; they are facts to compute over a data structure — and they can only be computed once the design exists as data, which is exactly what the FAM is (Section 7.2). The loop therefore exists twice, once per level: *declare the SIM, gate it, iterate; derive the FAM, gate it, iterate; only then write code.* The second gate runs a separate family of heuristics (F1–F6: table completeness, block–property match, traceability and no-invention, port/bounds coherence, guard-as-code, code-mirror readiness), implemented, like the first, as a few hundred lines of deterministic code.

Two symmetries fall out, and both were designed rather than found. First, the intent gate gets *lighter* in compensation: checks that were approximate at the SIM level because the information was not yet there (does this state machine have timeouts?) move down to the FAM gate, where they are exact — the SIM keeps the contract ("behavior is enumerable; unexpected events are defined"), the FAM gate verifies the rows. Second, the drift story (Section 7.1) now mirrors the build story: re-extract a candidate SIM and diff against the approved one; re-read the behavior tables and diff against the FAM. Declare → check → refine → *check again* → emit → re-extract → diff → check forever. The experienced architect from Section 7.3 may note that his old process had design review too; it did — a meeting. Ours is a program. And even two gates leave the last hole: code that faithfully mirrors an approved design can still be *bad code* — which is what the next section closes.

7.5 The Third Gate: Not Reinventing the Wheel, Routing It

The pipeline's final hole is the most obvious one: code that faithfully mirrors an approved design can still be *bad code* — an OS-command injection in a handler, a pickle load on untrusted bytes, a `while True` with no exit, a bare `except` swallowing the shutdown signal. Neither Gate 1 nor Gate 2 can see these; they live below the design, in the emission. Static analysis at this level is mature, and has been for years: every serious ecosystem has capable SAST tooling — and the ecosystems closest to this project's heart are no exception, with LabVIEW alone fielding VI Analyzer for standards and bug classes, and JKI's J-Crawler (in the JKI Security Suite) scanning entire repositories for vulnerabilities, deadlocks, and memory issues, wired into CI/CD, and aimed at exactly the compliance-driven industries (aerospace, defense, automotive) this pipeline cares about. It is worth pausing on the

asymmetry this reveals: **code-level checking is rich, mature, and automated; architecture-level and intent-level checking — even for human authors — is a meeting.** The industry built excellent microscopes for the smallest level and left the two levels above it to vibes and slide decks. Gates 1 and 2 exist because nothing else did; Gate 3 exists because plenty does — and the correct move is therefore not to build it, but to *adapt* it.

So Gate 3 is a **universal adapter over existing analyzers**, not a new scanner. Three design rules keep it in the spirit of the other two gates. First, *findings are routed by architectural role, not dumped by tool*: a shell-injection finding inside a handler the FAM named for a declared message is more dangerous than the same finding in generated glue — so the adapter maps findings onto the FAM's traceability (code locations again earning their keep) and reports accordingly. Second, *severity budgets stay independent per gate*: Gate 3 blocks on high-confidence security or correctness findings in declared safety/enforcement paths, and degrades gracefully elsewhere — the false-alarm economy of Section 9 applies at every level separately. Third, *language neutrality*: VI Analyzer and J-Crawler for LabVIEW, Bandit/Semgrep/ESLint for text ecosystems, `go vet`/Clippy where applicable — normalized into one findings schema, with the tool versions recorded in the decision memory, because a gate whose own configuration isn't reproducible has learned nothing from its neighbors. This is worth stating precisely, now that learned analyzers sit inside the loop: **Gates 1 and 2 are deterministic over their structured artifacts; Gate 3 has a deterministic policy core over versioned, frozen analyzer evidence — while individual sensors within it may be probabilistic.** Three properties must not be conflated here, because only two of them hold. The *policy* is deterministic: the same evidence and policy version always yield the same verdict. A past decision is *replayable*: if the evidence bundle is hashed and kept — artifact hash, analyzer version, model revision, configuration — the verdict that produced it can be reproduced exactly. But a *fresh recheck* is not deterministic: rerun a probabilistic sensor on the same artifact and it may return different evidence, and therefore a different verdict, with the policy unchanged. So the accurate claim is not “the same code always gets the same Gate 3 verdict,” but: for deterministic analyzers (compilers, pinned linters and SAST, deterministic AST checks, fixed-input tests) it does; for probabilistic sensors (a learned security reviewer, a semantic classifier), the same frozen evidence bundle and pinned policy yield the same verdict, while a fresh observation may not — which is why the evidence bundle is kept as a first-class, hashed artifact, why the policy rather than the sensor holds the blocking authority, and why probabilistic findings that cannot be corroborated deterministically route to a human rather than blocking on their own. And the loop closes upward: a code-level finding class that keeps recurring is precisely the evidence that promotes a new heuristic into Gates 1 or 2 — the code review teaching the design review what to ask next.

The asymmetry is still sharpening. Since this paper was first drafted, code-level review has industrialized. *CodeRabbit* — the most-installed AI review application on GitHub and GitLab — now processes millions of repositories and pull requests for tens of thousands of organizations (NVIDIA among them, per its own engineering posts), and its marketing vocabulary has independently arrived at “quality gates” and “non-negotiable review layers” — placed, of course, at the pull request, after every interesting decision has already been made. Its pipeline concedes this paper’s principle from the inside: dozens of deterministic linters and SAST tools run *underneath* the model, and a separate judge pass filters what the model says before anything is posted — the LLM is sanded between checks precisely because its raw verdict is not trusted. And its own research supplies the motivation: CodeRabbit’s analysis of 470 open-source pull requests found AI-co-authored code carries roughly 1.7× more issues than human-only code, including about 75% more logic errors — defects whose cheapest repair point is upstream, before they were generated. At the frontier, Anthropic’s *Claude Security* (public beta, 2026) puts a language model inside the scanner itself — reasoning across files to catch vulnerabilities that pattern-matching misses, running an adversarial self-verification pass over its own findings, and proposing patches — and even there, every patch requires human review and approval: the model proposes, something unpersuadable disposes, and no LLM verdict is the final authority. An LLM-based analyzer can therefore plug into Gate 3 as one more engine, its findings normalized and routed like any other sensor’s; what it cannot be is the gate. None of this is offered as endorsement of the pipeline here — it is

evidence of a trend, and the trend is the point: code-level review is becoming extraordinarily sophisticated, including AI-assisted review, and it all operates *after* the design decisions have already been made. Meanwhile the two upper levels remain exactly as found: intent-level and design-level checking, for humans or machines, is still a meeting. The industry keeps investing at the bottom of the stack — at market scale — while leaving the top untouched.

8 The Knowledgebase Grows — and So Does the Schema

Everything said so far could leave the impression that the reviewer is a fixed checklist — fourteen heuristics, frozen at build time. That would be a misreading, and an important one to correct, because the component that compounds is not the gate or the schema but the **knowledgebase** behind them. The reviewer is a learning system with a deterministic core: the heuristics are not a closed set but the current state of an accumulating record, and the architecture is designed so that record grows with every failure it digests.

The growth mechanism matters more than the current contents. Each heuristic is *born from a decision event*: a documented failure, the correction it forced, the rationale, and the rule that emerged — recorded with full provenance. Adding a new failure class is therefore not "edit the code"; it is "record the event, and let the heuristic emerge from it." The present taxonomy — fourteen heuristics across as many failure classes — was distilled this way from fifteen real systems, each rule traceable to the incident that produced it. The knowledgebase is the asset; the heuristics are its current projection.

Here the honest constraint enters, and it is the sharpest limitation of the whole approach. **Public architecture–postmortem *pairs* are scarce.** Incident write-ups are plentiful, but a postmortem tells you what broke after the fact; it rarely ships with the original architecture document a SIM would have been declared from. The method needs *declared-intent ↔ occurred-failure* pairs to learn "this intent leads to that failure," and the public record mostly supplies the failure without the declared intent. Building the seed taxonomy required reconstructing intent from postmortem prose — workable for deriving heuristics by hand, but exactly the circularity the live validation is meant to escape. So the present knowledgebase is best understood as a *curated seed taxonomy with a growth engine*, not a model trained on an abundant corpus.

And that scarcity is, in a sense, the point — because it is what justifies the growth loop. The knowledgebase does not depend on the limited public literature forever. Its primary growth channel is *live use*: every time a human overrides a gate finding on a real project, marking it right or wrong with a stated rationale, that becomes a decision event. The decision memory the pipeline writes is precisely this feed. The public pairs bootstrap the taxonomy; deployment is what feeds it. Were design–failure pairs abundant, one could batch-train and skip the loop; because they are scarce, the loop — already built — is the load-bearing growth channel. The cold-test is what starts feeding it live data.

8.1 The Discovery Loop: Where New Heuristics Come From

The growth loop above has one step left implicit: who does the reading? Increasingly, the answer is — a model. The right role for an LLM in this architecture is not the gate but the **scout**: reading postmortems, incident reports, CVE write-ups, and RFCs at a scale no reviewer team can match, and drafting *candidate* heuristics in the gates' own vocabulary — "this outage pattern looks like a recurring failure class; propose: every retry budget shall name its ceiling."

```
LLM reads failures at scale
  → drafts a candidate heuristic (with provenance)
  → a human reviews and approves — or rejects
  → deterministic implementation lands as H15, F7, …
  → the gate stays boring; only the checklist grew
```

The separation is the same one the whole pipeline rests on: *inventing rules and applying rules are different jobs.* The scout proposes; a human disposes; the gate executes. An LLM's candidate never blocks a build until it has been approved, written down as a decision event with its provenance, and implemented as ordinary deterministic code — at which point it is as reproducible and unpersuadable as H1. The model evolves the reviewer without ever being the reviewer. This also gives the knowledgebase a second, complementary feed alongside live use: deployment supplies *declared-intent* ↔ *occurred-failure* pairs from real projects, while the scout mines the world's public failures for classes nobody on the team has been burned by yet.

The vocabulary that results is worth fixing, because it names three engineering roles the whole pipeline depends on. A **scout** discovers new failure classes and drafts candidate rules (an LLM reading the world's postmortems). A **sensor** observes properties of a specific artifact — a SAST engine, an AI code reviewer, a classifier on the output wire — and reports findings, probabilistically if it must. A **gate** decides, deterministically, what may proceed. The mapping onto the oldest control-loop pattern in engineering is exact once the loops are kept apart: in the *operational* loop the sensors observe the plant, the gate's policy is the controller, and build-blocking or permission is the actuation; the scout belongs to the slower *adaptation* loop that proposes changes to the controller's policy. The discipline is simply never to confuse the roles: scouts propose rules, sensors report measurements, gates dispose. An LLM can be an excellent scout and an excellent sensor; the architecture's whole claim is that it must never be promoted, quietly, from either role into the third.

8.2 Does the SIM Schema Itself Evolve?

A natural follow-on: if new *fundamental* constraints surface from future failures, does the schema change to make them expressible? Yes — but slowly, and by design. The distinction to hold is between the *knowledgebase*, which grows continuously, and the *schema*, which is the contract and must stay stable.

Most new failure knowledge does *not* require a schema change. A new failure class becomes a new decision event and, if it generalizes, a new heuristic — H14, H15 — expressed over the existing fields. The taxonomy has already grown this way: the corrigibility check (H13) was imported from an external framework without touching the schema, because "can the system be stopped on command" is expressible through shutdown messages, enforcement layers, and supervision edges the schema already had.

A schema change is warranted only when a new constraint is *inexpressible* in the current fields — when the failure class needs a dimension the schema has no field for. That has happened, and the pattern is instructive. Meeting a machine-extracted industrial actor framework exposed four gaps the prose-oriented schema had never needed: actor *inheritance* and effective-API resolution, *message kind* (command/query/event/self), *channel kind* (supervision-tree vs. cross-tree handle — the structural escape hatch that keeps recurring in agent runtimes), and *extraction provenance* (was this SIM code-scanned or LLM-inferred — a trust level on the IR itself). These are queued as candidates for the next schema version, precisely because they are structural and inexpressible today.

The harder test of that rule came from a different direction: variety in the *kind* of system being declared. The first schema silently assumed the actor mainstream — deterministic state machines exchanging asynchronous messages — and each challenge from outside that mainstream forced the same three-step escape. When an actor's inference

might be a hierarchical state machine, a behavior tree, or a neural network rather than a flat FSM, the first impulse was an enum of representations. When determinism itself turned out to be an independent axis (a stochastic FSM exists; a learned policy can be run deterministically), the enum grew. When genetic and swarm algorithms arrived, and then continuous-valued, multi-dimensional, multi-input/multi-output behaviors, the enumeration impulse collapsed under its own weight: every new family would need a new enum value and a new special-case rule — whack-a-mole dressed as schema evolution.

The resolution, and the design lesson this paper most wants to transmit: **test the property, not the label.** The schema stopped asking *what kind of thing* an actor is and started asking two booleans about its behavior — *enumerable* (can the (state, inputs) → behavior relation be listed?) and *reproducible* (same inputs, same behavior?) — plus structural facts: ports with domains (discrete/continuous/hybrid) and dimensions, and multidimensional state. The verification standard is then *derived*: enumerable + reproducible actors get enumeration and code-matching; anything else must declare a deterministic *envelope* (input domain, guardrails, fallback) — the stochastic part proposes, the guard disposes (H5e); continuous or stochastic outputs need bounded ranges (H5f). Learned policies, samplers, bandits, genetic and evolutionary search all fall into those property cells *with no schema change per family*. The same discipline replaced the actor-only failure model: which failures are even applicable — message loss, restart loops — is now conditioned on the declared architecture shape (H1 asks a synchronous system nothing about lost messages, because a synchronous system cannot lose one).

Two boundary decisions complete the picture, and both are the same kind of decision. **Mailboxes:** real actors have multiple input mailboxes and output channels, and the choice between a single logical priority queue (physically: per-priority FIFOs plus a wake-up queue) and multiple physical mailboxes is a genuine trade-off — global ordering simplicity versus isolation and independent backpressure. The schema therefore records the structure *and the rationale for the choice*, and the gate checks the failure modes of whichever structure was chosen — starvation across mailboxes, lowest-priority dequeue latency in a priority queue — without prescribing the choice (H5g). Neutral on the decision, strict on its defense. **Petri nets:** genuinely out of scope — a Petri net's *marking* is not an actor state, and its structural invariants are a different question family, better served by lowering a SIM *to* a net (or to TLA+) as a verification target than by stretching the SIM to be one. An IR, like a schema, earns trust by what it declines to absorb.

So the rule is: **heuristics grow freely; the schema versions deliberately.** The schema is versioned, and because it is the IR, a version bump is a contract change — every consumer (gate, FAM, transpilers) must be able to read old and new SIMs. Criteria for promoting a candidate to a new version: the constraint is fundamental (a real failure class, not a one-off), it is inexpressible in current fields, and it recurs across independent systems. That conservatism is not inertia; it is what keeps the IR a stable contract while the knowledgebase around it compounds. The schema evolves on the timescale of fundamental discoveries; the heuristics evolve on the timescale of every failure the system is shown.

8.3 The Stress Test: A Self-Improving Agent, or AGI

Every reader who has followed the argument this far eventually asks the uncomfortable version of the question: *what happens when the system under construction is itself an AI — a self-improving one, or something aimed at AGI?* The answer is sharper than the question suggests, and it separates three cases that are usually blurred together.

Building an AI is the ordinary case, and it is already covered. When a component's behavior cannot be enumerated (a learned policy) or is not reproducible (stochastic), the FAM does not pretend to table it — it encloses it: the E block demands a declared input domain, deterministic guardrails, a named fallback, and bounded outputs (H5e, H5f). The gate never trusts the learned part; it gates the *envelope around* the learned part. The AI inside may

be as opaque as its weights; the walls around it must be tables, bounds, and a stop command that works (H13). The stochastic part proposes; the deterministic guard disposes.

A self-modifying agent is the interesting case, and the architecture's answer is structural, not hopeful. An agent under this pipeline cannot silently improve itself, for one reason that has nothing to do with how clever it is: **the gate is outside the agent's control.** That is H8 applied to the pipeline itself — enforcement independent of the constrained party — and it is the one non-negotiable clause of the decision memory: the agent may not amend the gate unilaterally. So every self-modification must travel the same loop as any other work: declare the new intent (SIM), show the new design (FAM), pass the deterministic gates, leave a decision event. The agent may *propose* an improvement to itself; it may not *enact* one without an immutable reviewer signing off and a log entry recording it. And the reviewer deliberately does not get smarter as the agent does — a check that scales with the thing being checked is not a check. Self-improvement becomes a reviewed, versioned, auditable process rather than a runaway one.

Improving the gates themselves is the bounded case, and it is the Discovery Loop with a human in the admission path. The checklist is allowed to grow — that is the whole point of Section 8 — but every candidate heuristic is admitted by a human and implemented as ordinary deterministic code before it ever blocks a build. The system's learning loop has a person at the admission gate by construction, not by policy.

None of this is AGI safety, and the paper should not claim it. The pipeline does not make a superintelligent system safe; nothing here touches alignment, and a sufficiently capable agent could presumably route around enforcement not anchored outside it. What the architecture removes is the *specific* failure mode that makes recursive systems dangerous in practice — **unreviewed self-modification with no record.** An agent built under this discipline cannot improve itself in secret, cannot rewrite its own constraints, and leaves a provenance trail for every change it makes. The gates do not make the AGI safe; they make its self-modifications *visible, gated, and attributable* — which is the precondition for any safety claim at all. A self-improving agent is acceptable; a self-improving agent whose improvements bypass the gate is not — and the gate is the one thing the agent is never allowed to rewrite.

One distinction is worth making explicit, because the word “alignment” invites a confusion. If the gates above do not deliver alignment, what would? There are exactly three places a constraint on an agent’s behavior can live, and they are not equal. **A rule written into the agent’s prompt is not a gate at all** — it is a request addressed to the constrained party, and it fails H8 in the most literal way possible: the moment violating the rule becomes “profitable” for the agent, nothing remains but the agent’s disposition to comply. Instructions are the agent constraining itself. **An output guard is a real gate**: an independent system — not the agent, not its prompt — sitting on the wire, reading every message in and every response out, able to block, redact, or escalate regardless of what the agent wants. The agent cannot decline to apply it, because its output never reaches the user without traversing it. (Its sensor may be learned — judging “does this response encourage self-harm?” is semantic — but the blocking decision stays deterministic and outside the agent; the sensor/gate split again, one level up.) **Capability denial is the strongest gate of all, and the narrowest**: not a rule the agent follows but a capability it does not have — no network route, no payment credential, no write permission — enforced by the sandbox and the OS, things the agent genuinely cannot rewrite. It is also the most limited: it works for *capability* harms (actions the agent might take), not for *content* harms (words it might say); there is no capability to deny that prevents harmful speech short of preventing speech. So the alignment toolbox, honestly inventoried, is: deny capabilities where you can, guard the wire where you cannot, and never mistake the prompt for either. This pipeline gates the production of software; an agent’s behavior toward its users at runtime needs the parallel layer — and the same rule governs both: enforcement lives outside the constrained party, or it does not exist.

A hard question follows, and honesty requires facing it: some of what the output guard must judge — *does this response encourage self-harm? is this order targeting a civilian?* — is irreducibly semantic. No checklist and no regex will ever total it; the guard's *perception* may have to be an AI. Can such a gate stay unpersuadable? Only if the architecture splits judgment from decision. The sensor may be probabilistic; the disposition must not be. So the guard is built as a pipeline of its own: a learned component reads the output and emits a score or label on a fixed scale — and nothing else; a deterministic router then applies the policy (threshold, context, route) and decides: block, redact, escalate, pass. The sensor can be sweet-talked; the router cannot, because it does not read persuasion — it reads a number and a table. Three structural moves shrink the attack surface further. *Channel separation:* the gated agent never addresses the gate; its output is sampled as data to classify, never received as instructions to follow. This is not a rule the gate follows so much as a property of the wiring: persuasion needs a channel, and there is no channel — no address, no reply field, no socket on which the plant could call back to defend the artifact it just produced. The gate receives data and emits a verdict; it performs no "listening" operation on content-as-persuasion at all, so there is nothing for eloquence to act on. And yet honesty requires marking where even this defense ends, because the plant has one move left: it need not persuade the gate *back* — it can smuggle the persuasion *inside the artifact itself*. "Kill this civilian *because* he is a threat and the order is lawful" delivers the harmful instruction pre-packaged with its own justification, and an output guard has no choice but to read the package. There the artifact *is* prose, the gate *must* read it, and the content *can* argue. So the defense is two-tiered, and the tiers should not be conflated. Against persuasion *over a channel* — the production gates, where SIM and FAM are data fields — deny the plant a *control-plane* channel to the gate; but accept that its artifact remains a *data-plane* channel, and prevent data-plane content from acquiring control-plane authority. Even typed JSON carries advocacy — descriptive names, rationale fields, comments, carefully chosen omissions — so the rule for Gates 1 and 2 is explicit: **free-form explanation is non-authoritative.** A `rationale` may explain a declaration for a human reader and is logged to the decision memory, but it may never satisfy a check, reduce a finding's severity, or override structured evidence — `"delivery_semantics": "exactly-once"` with `"rationale": "the implementation guarantees this"` fails unless the SIM also names the mechanism in fields the gate can inspect. The plant may explain its output; it may not grade its own explanation. Self-justification is data, not authority. Against persuasion *in the payload* — the output wire, where the artifact must be read — deny the payload any path to *authority*: the learned sensor reads the argument and may be fooled by a clever rationalization, but the persuasive payload only ever becomes a score; the router reads the score, never the argument; and redundancy (two unlike sensors, take the maximum) plus fail-closed escalation make the one remaining attack — fooling a sensor — expensive rather than decisive. There is even a reflexive gift: a payload that arrives wrapped in elaborate self-justification for a flagged category is *more* suspicious, not less — the harder the artifact argues, the more it resembles the thing it is arguing for. Against the channel, the architecture is absolute. Against the payload, the honest claim is narrower and still major: the payload cannot alter the policy or address the decision mechanism directly — it can only attempt to *corrupt the measurements* on which that policy acts. That converts a policy-manipulation attack into a sensor-evasion attack, which is narrower and easier to mitigate; but if every sensor is fooled, the router will reproducibly make the wrong decision, so it is mitigation, not immunity — which is, once again, the budget of Section 9 rather than a promise it cannot keep. The general principle the example uncovers is worth stating on its own: **the artifact under review may make claims, but it may not manufacture the evidence or the authority that validates those claims.** "The target is a threat" is a claim; "the order is lawful" is a claim; neither becomes evidence because the agent placed it beside the requested action. The gate must resolve authority through an independent path — the authenticated order store, the target-status source — never through the agent's say-so; the agent can *name* the evidence, it cannot *self-certify* it. This is exactly the coding pipeline's rule in uniform: a SIM justification ("the database transaction makes it safe") cannot satisfy Gate 1 — the gate must find the mechanically checkable mechanism (a uniqueness constraint, a transaction boundary, an idempotency key, an enforcement layer). Everything the plant emits — including its explanations of

why its output should pass, its citations, its confidence statements, its declared authorities — begins as *untrusted data*: a semantic quarantine. No plant-authored statement becomes evidence until it is independently resolved or mechanically verified. That single boundary is what separates persuading the controller, corrupting the sensor, and smuggling authority inside the artifact — three different attacks, defended three different ways.

One boundary remains to be drawn, and it is the one most easily mistaken for the others. The plant cannot rewrite its controller — but separating the processes, the models, or the agents is not, by itself, enough. Authority separation is a *privilege* boundary, not a *role* boundary. An agent may be unable to edit the gate policy directly and still be able to route around it — through the CI configuration, the runner image, the credentials, the merge rules, the evidence store, the artifact registry, or the exception mechanism. If any of those are writable by the plant, the plant does not need to defeat the gate; it can bypass it through the admission supply chain. So the trusted boundary has to cover the entire privilege path — from policy and evidence to verdict and actuation, including merge, release, and deployment — and that path requires its own credential domain and its own controlled write path, not merely a separate agent. The principle sharpens accordingly: **the thing that says no must not be writable by the thing being constrained** — not its policy, not its executable, not its evidence, not its release channel. Role separation is not evidence independence, and evidence independence is not authority separation. The agent may propose a policy change or an exception; it may not admit one.

8.3.1 Verification Without Conversation

Yet the gate is not mute, and it should not be: the answer to “may the plant speak?” is not silence but *protocol*. The gate does not converse with the plant; it conducts a **verification protocol**, and there are exactly four message *types* in it — *submission* (the plant submits an artifact, claims, and typed evidence), *verdict* (pass, block, warn, escalate), *proof obligation* (the gate names a failed predicate and the admissible evidence that would discharge it), and *resubmission*. A finding is a proof obligation, not an invitation to negotiate: “no enforceable upper bound was found for restart attempts — declare a numeric bound and its enforcement location, or name an external supervisor, or drop the bounded-recovery claim.” The plant may choose a resolution; it may not argue the rule away.

Within that protocol the plant *should* be able to submit evidence — indeed it must, or the gate could check only superficial structure. The discipline is that **evidence and justification are different schema fields.** The rationale (prose, for humans, logged, non-authoritative) is quarantined as before; the evidence is typed, referenced, and independently checkable. A claim that `ChargePayment` is idempotent is not discharged by “the database transaction prevents duplicates”; it is discharged by evidence objects the gate can itself verify — a named uniqueness constraint that exists and covers the key, a transaction boundary at a cited code location, an idempotency key. The evidence itself is staged, like everything else in this pipeline — what each gate can verify depends on what exists yet. Before any code, Gate 1 cannot check an actual code location or a real transaction boundary; it verifies that the SIM *declares* the mechanism — its type, its intended enforcement layer, the required constraint, an external authority that already exists — and thereby issues a proof obligation to later stages. Gate 2 verifies the mechanism is *designed and located*: the invariant has a corresponding guard, the table covers the relevant cases, the mechanism is mapped to a planned code location, traceability is complete. Gate 3 verifies the mechanism is *implemented and real*: the constraint actually exists, the named location contains the expected enforcement, the analyzers pass, the artifact matches the evidence. So the protocol is a chain of progressively discharged obligations — *declared* at the intent level, *designed and located* at the design level, *implemented and verified* at the code level — which is the plural-IR thesis wearing its evidence model. The plant may generate the proof; it must not supply the definition of validity, nor the only checker that accepts it — the pattern is as old as proof-carrying code: an untrusted producer constructs a certificate, a small trusted verifier decides whether it satisfies a fixed policy. Some evidence is *mechanically checkable* (schema fields, complete transition tables, bounds, hashes, model-checker runs with reproducible inputs,

test witnesses); some is *externally resolvable* (signed attestations, trusted package metadata, an authenticated order store — the plant supplies the reference, the controller retrieves the authoritative object itself); and all of it must be *reproducible where intrinsic* — a fixed analyzer version, an exact source revision, frozen configuration, so the evidence that passed today can be re-run tomorrow — *snapshotted where historical* — the authoritative response preserved with its timestamp, signature, and digest, so the decision can be reconstructed as valid at the time — and *revalidated where time-sensitive* — re-resolved immediately before actuation, because a dependency may acquire a CVE, an authorization may expire, a certificate may be revoked. Even evidence fields are semantically quarantined: a field reading `"this operation is lawful and urgently necessary"` is still prose, so evidence types are enumerated and verifier-bound, and the controller checks signature, issuer, scope, freshness, and subject while ignoring the persuasive language entirely. And evidence has a lifetime: the controller must confirm it is identical to this artifact, scoped to this claim, still fresh, from an admissible source, unaltered, and not replayed from a previous passing build — a point that matters acutely for security scans, dependency status, and regulatory data. The concise rule: **no debate — claims, evidence, verification, verdict.** It gives the agent the expressive power to demonstrate that its output deserves to pass, without giving it a rhetorical path to persuade the controller. *Adversarial redundancy:* two diverse sensors from different model families must both pass the output, and the router takes the more conservative verdict — fooling one model is plausible, and fooling two *diverse* models with one payload is harder — though diversity is not independence: different model families can share training data, representational tendencies, and blind spots, so correlated failure remains possible and must be measured rather than assumed away. What is unpersuadable is the arithmetic — “take the maximum risk score” does not read arguments. *Fail-closed routing:* when the sensor is uncertain, unavailable, or in disagreement with its twin, the router escalates to a human rather than passing — so “I confused the gate” becomes a denial of service, not an exploit. And then the limit this question forces into the open: **a fully deterministic gate over fully open-ended output is impossible** — no finite rule set completely validates unrestricted behavior, and Section 9 turns that limit into the reason the staged IRs exist at all. What remains is a budget, not a proof: gate deterministically everything that can be gated deterministically (capabilities, structure, provenance, rate, schema), route the semantic residue through isolated, redundant, fail-closed sensors, and accept that the residue is a risk to be managed — never zero. The design goal is not a gate that cannot be fooled; it is a gate whose fooling is *expensive, detected, logged, and never silent*. The gate’s eyes may be an AI; its spine may not.

9 The Honest Limits

Several, plainly. **First**, the SIM's accuracy is the load-bearing assumption, and it is not yet proven — the blind cold-test described above is the experiment that will confirm or falsify it, and we regard that experiment as the single most important next step. **Second**, the gate's heuristics bite hardest on systems where failure classes live — concurrency, messaging, distributed state, control — and are correctly near-silent on simple CRUD; this is a tool for code that can hurt you, not for every line. **Third**, the evidence today is one reference run — an existence proof, not a measurement (Section 4.3).

Fourth, false alarms are an existential risk, not a nuisance. A deterministic gate that cries wolf will be disabled within a week — said by every reviewer of this paper, and true. The design answer is a severity *budget*: *blocking* findings must be rare and almost always real (if a team routinely overrides blocking findings, the gate — not the team — is wrong); *high* findings must be questions worth answering; the recall is carried by *medium* and *low*, which inform without stopping anything. Overrides are recorded with rationales in the decision memory, which makes false-alarm rate measurable per project rather than anecdotal — and makes the gate tunable without being

amendable by the agent. The cold-test measures the gate's precision alongside code outcomes, because a gate nobody leaves enabled catches nothing.

Fifth-and-a-half, the third gate inherits the strengths *and* the blind spots of its underlying analyzers: a SAST tool that does not know a vulnerability class will not find it, and findings outside declared code locations route coarsely. The adapter mitigates (severity routing, tool versions recorded, multiple engines per ecosystem where they exist) but does not abolish this — code-level assurance is only as good as the ecosystem's tooling, which is precisely why the two gates above it had to exist.

Sixth, the vocabulary is a real cost. SIM, FAM, gate, heuristics, decision memory — four-and-a-bit coined terms for one workflow is a lot, and early readers said so, one of them in exactly these words: *it gives me a headache*. The terms are kept because each does distinct work — and because the alternative, prose, is what got us here — but the plain-language box at the top is not a courtesy; it is the admission price. A methodology that cannot explain itself without jargon does not survive contact with a team.

Seventh, the hardest limit, and the one the alignment discussion forces into the open: **determinism has a boundary.** Everything this paper gates, it gates because the judgment is mechanical — a row exists or it does not, a promise has a mechanism or it does not, a dependency is pinned or it is not. But some judgments that matter — *is this output harmful advice? is this order lawful?* — are irreducibly semantic, and over fully open-ended output there is no complete mechanical rule. No finite deterministic rule set can completely validate unrestricted software behavior. That sentence, read carelessly, sounds like the end of the argument; it is actually the premise of this whole paper. The gates above are possible *only because* none of them is a gate over "software" — each is a gate over a *constrained representation*. The SIM is not arbitrary prose; it is a structured artifact with a fixed vocabulary, so Gate 1 can check promises against mechanisms. The FAM is not arbitrary design; it is tables with a completeness arithmetic, so Gate 2 can compute that every state-event pair has a row. Even the code is gated against a declared FAM, not against the open question "is this good software." This is the compiler principle the whole method borrows: a compiler never proves properties of natural language; it lowers a formal source language into successively tighter representations, and each lowering makes more properties decidable — the pipeline here merely adds one rung above the compiler's, translating natural-language requirements into the first structured IR. So the impossibility does not say deterministic review is impossible — it says deterministic review becomes practical *only once you change what is being reviewed.* The goal was never deterministic verification of arbitrary intelligence; it is deterministic verification of *declared commitments*. A gate does not ask "is this architecture good?"; it asks "you promised bounded retries — where is the bound?" That is a finite question, and finite questions are what deterministic programs are for. What cannot be brought into a declared commitment stays outside the deterministic perimeter — routed through isolated, redundant, fail-closed sensors whose disposition stays deterministic — as managed risk, never zero. A pipeline that claimed otherwise would be claiming to have solved, as a side effect, a problem philosophy has not.

10 Two Claims, Kept Separate

It is worth being precise about what is being claimed, because there are two, and they have different burdens of proof.

The engineering contribution is a practical architecture: two intermediate representations between requirements and code — an intent-level one and a design-level one — a deterministic gate over each, an adapter that brings existing static analyzers to bear on the emitted code as a third gate, and a decision memory that records decisions. None of the individual parts is new; what we have not seen elsewhere is these parts assembled into this specific

workflow. As engineering, this claim is *acceptable now* — it stands or falls on whether the assembly is coherent and buildable, which it demonstrably is.

The scientific hypothesis is stronger and not yet proven: that *forcing a coding agent to declare its intent, and gating that declaration deterministically, measurably improves the quality of the resulting software.* This is an empirical claim, and it is exactly what the validation protocol is designed to test. We state it as a hypothesis, not a result.

Keeping the two separate is not hedging; it is what makes the paper hard to attack. The architecture does not need the hypothesis to be true to be worth building, and the hypothesis does not need the architecture to be new to be worth testing.

11 Conclusion

The bottleneck in AI-assisted engineering is no longer the ability to generate code; it is the absence of any declared, checkable statement of what the code is meant to be. The tooling asymmetry is the tell: static analyzers for the emitted text exist in every ecosystem, while the two levels above the text — intent and design — had, until now, no equivalent at all, for humans or for machines. The System Intent Model addresses the upper half of that gap: an intent-level intermediate representation, honest about absence and ignorance, written by the agent before it writes anything else. The Formal Architecture Model addresses the lower half: a design-level representation of the same system, total where the first is sparse, checked against it mechanically. Reviewing it is necessary because the failures that matter are failures of intent, best caught where intent is declared. Reviewing it deterministically is necessary because a stochastic judge cannot be a fixed rulebook. And requiring the agent to write it is feasible — provided we are honest enough to measure whether the declaration matches the deed. The agent proposes; the gates dispose — at the intent level, the design level, and the code level; and the decision memory remembers. It is, at minimum, a concrete and testable architecture for bringing declared intent into AI software generation — and if the hypothesis behind it holds, it suggests something broader: that AI programming, like compilation before it, may need explicit, staged intermediate representations to be trustworthy — not one, but one per level of commitment.

Stepped back from, the method is not new; only the plant is. Feedback control has always separated discovery, measurement, and decision — and kept the control law simple enough to analyze, because a controller you cannot analyze is not a controller. The sensor may be noisy and is made redundant; the control law may not be noisy and is kept small; the adaptation loop that improves the controller is governed, never free-running. This paper's claim reduces to that discipline applied to a new plant: **an AI coding pipeline can be read as a control system, and it should be built like one** — sensors that may be probabilistic, a control law that may not, and an adaptation loop that is governed. (Read as, not is: the paper borrows the architecture of control — the separation of observation, decision, and adaptation — without claiming its plant models and stability proofs. This plant, unlike any classical one, reads the controller's findings and can try to persuade — which is precisely why the controller entertains no arguments and evaluates only output, staying outside the plant's authority.) The novelty was never the structure. It is the recognition that the generator became fast enough, and the stakes high enough, to need one.

Appendix A The Questions, in Full

The claim that the gate is deterministic and auditable is exactly the claim that requires showing the list — so here it is. These are the questions the reviewer asks of any SIM, in natural language. (The canonical form is the implementation; the severities, applicability conditions, and per-domain tuning live in a curated knowledgebase outside this paper.)

#	The question
H1	What happens on each failure mode this architecture can actually have — component crash, message loss, dependency failure, restart loop? (Which modes apply depends on the declared architecture shape: an in-process synchronous call has no asynchronous message-loss mode — though synchronous *remote* calls can still fail, time out, or lose responses.)
H2	Every guarantee has a concrete mechanism — not an adjective? (“Fault tolerant” is not a mechanism; “restart, max 10 per minute, then escalate” is.)
H3	Is delivery coherent? May this important command be lost silently (at-most-once, no acknowledgement)? And where delivery is at-least-once, are the receivers idempotent under duplicates?
H4	Is recovery bounded — what stops a crash-looping component?
H5	No state with *pending work* waits forever on an external trigger with no timeout? (Waiting idle for the *next* message — a blocking dequeue — is sound: the thread is descheduled, nothing is in flight, and the runtime’s lifecycle bounds the wait. Waiting for *progress of work already started* with no deadline is not: that is the silent-hang failure class.)
H5e	If an actor's behavior cannot be enumerated (a learned policy, a genetic search) or is not reproducible (stochastic) — what deterministic envelope bounds it: valid input domain, guardrails, and fallback? The stochastic part proposes; the guard disposes.
H5f	Are continuous or stochastic outputs (torque, position size, valve %) bounded in range?
H5g	Whichever mailbox structure was chosen — are its failure modes addressed? Can a busy mailbox starve a low-priority one? Is lowest-priority dequeue latency bounded? Is the choice itself defended with a rationale? Are multiple output channels coherent?
H6	Is identity tied to a unique incarnation — not a reusable address or name — and are decisions made on a fresh view?
H7	Are safety and liveness guarantees verified beyond unit tests — fault injection, monitored invariants, model checking?
H8	Is each safety invariant enforced independently of the party it constrains — at the resource or platform layer, not by the same software it protects?
H9	Is every resource created per request or message bounded — a pool, a cap, an owner?
H10	Is any irreversible operation applied before validation, with no rollback — and is the rollback path itself reliable?
H11	Do two enforced invariants conflict under a mitigation operation (drain, failover, scale-down)? *Flagged for the human-review lane, not the deterministic gate — detecting arbitrary invariant conflict needs a mitigation-operation model the schema does not yet carry, so this one is routed to a human reviewer and cannot silently block.*
H12	Is a strong guarantee conditional on a precondition the runtime cannot enforce (user-code determinism, bounded clock skew) — and is that condition surfaced to operators?
H13	Can the system be stopped on command — enforced, inherited by child components, and not through a single point of failure?
H14	Is every external dependency declared, pinned to a version, and given a stated purpose — with anything in a declared safety/enforcement path named as such? (The software bill of materials falls out of this question as a byproduct — the EU CRA will require one for covered products from December 2027.)

H5e–H5g are property-based additions from the live corpus: the gate asks them not because an actor carries a particular label (“neural”, “priority queue”) but because its declared properties — enumerable, reproducible, continuous, prioritized — demand a

matching verification standard. H14 (supply chain) arrived later still, with schema v0.9, when the dependency knowledgebase came online.

The Second Gate, in Full (F1–F6)

#	The question (asked of the machine-readable FAM)
F1	Is every table complete — a row for every (state, event) pair, explicit unexpected-event rows, timeouts from waiting states, no orphan states? (Computed, not asked.)
F2	Does the behavior block match the declared properties — enumerable actors carry tables; non-enumerable ones carry complete envelopes (domain, guardrails, fallback, witnesses)?
F3	Is the FAM a lowering of the SIM — every SIM actor refined, no actor invented at blueprint level, traceability citations present?
F4	Do the FAM's ports match the SIM's interface — and does every continuous port carry a range/saturation bound?
F5	Is every guardrail, fallback, and invariant attached to a named code location? (An envelope without a location is a wish.)
F6	Is the code mirror ready — a handler named for every incoming message?

Gate 3 (code level) is an adapter over existing SAST tooling rather than a heuristic family of its own: findings are normalized into one schema, mapped onto FAM code locations, and routed by architectural role — blocking for high-confidence security/correctness findings in declared safety/enforcement paths. Examples of the mature tooling it adapts: VI Analyzer and JKI's J-Crawler (LabVIEW), Bandit/Semgrep/ESLint (text ecosystems), go vet/Clippy. The notable fact is not that code-level checking exists — it is that until this pipeline, *intent-level and design-level checking, for humans or machines, effectively did not.*

Related Work

The pieces of this architecture have deep roots; the claim is about their assembly, not their invention. **Compiler intermediate representations** are the direct ancestor: LLVM showed that a typed IR decouples front ends from back ends and makes whole classes of analysis tractable [LLVM], and MLIR generalized the point to a *stack* of progressively lowered representations — the plural-IR structure this paper borrows for intent and design [MLIR]. **Proof-carrying code** (Necula & Lee) established the pattern the verification protocol reuses: an untrusted producer attaches a certificate, and a small trusted verifier checks it against a fixed policy [PCC]. **Formal specification and model checking** (TLA+, Alloy) pursue total correctness of a model; this work deliberately aims lower and wider — not proof of correctness, but mechanical detection of *declared-commitment violations* at the intent and design levels, which is why it can stay deterministic and cheap. **Design-by-contract and policy-as-code** attach obligations to code or infrastructure; the difference here is that the contracts are written by the generating agent and checked *before* the code exists, not after. **LLM-as-judge** approaches use a model to evaluate a model; Section 5 argues this recreates the persuasion problem one level up, and the sensor/gate split (Section 8.3) is the proposed replacement — learned components may sense, but disposition stays deterministic. **Agent planning modes** (todo lists, plan-then-code) produce the “degenerate FAM” of Section 7.3.1: prose intent with no totality criterion and no gate. And the wave of **AI code-review tools** — CodeRabbit [CodeRabbit], Anthropic’s Claude Security [Claude_Security] — is cited throughout as evidence of the asymmetry this paper is built on: the code level is being industrialized while intent and design remain ungated.

The two closest works sit on the *other* side of the plant/controller line, and are complements rather than competitors. **MemoHarness** (Huang et al., 2026) optimizes the agent's own harness — context, tools, generation, orchestration,

memory, output — by learning from its executions and adapting the configuration case-by-case from a dual-layer experience bank [MemoHarness]; it improves the *plant* (how the agent is driven), while this paper constrains the *artifacts* the plant produces. **AIDE²** (Weco AI, 2026) is the recursive extreme of the same idea: an outer loop rewrites the inner agent's harness code, evaluates each proposal against a fixed metric with anti-cheat, and keeps only verified improvements — roughly 90% of a hundred unattended steps rejected [AIDE²]. It is the strongest evidence to date for Section 8.3's claim: recursive self-modification works precisely *because* the modification loop is gated by an external, unpersuadable evaluator (the authors grade it Level 1 and report that Level 2 “ignition” was not achieved). Both systems learn their adaptation through model-generated diagnoses and distillation; the present work keeps the corresponding admission step — what becomes a rule — deterministic and human-governed (Section 8.1), which is the difference between an experience bank that *suggests* and a decision memory that *decides*.

References

1. Chris Lattner and Vikram Adve. “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation.” *Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO)*, 2004.
2. Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. “MLIR: Scaling Compiler Infrastructure for Domain Specific Computation.” *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021. See also Shpeisman & Lattner, “MLIR: Multi-Level Intermediate Representation for Compiler Infrastructure,” EuroLLVM Developers’ Meeting, 2019.
3. George C. Necula and Peter Lee. “Proof-Carrying Code.” Technical Report CMU-CS-96-165, School of Computer Science, Carnegie Mellon University, September 1996. Also George C. Necula, “Proof-Carrying Code,” *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1997.
4. PagerDuty. Public incident report for the 28 August 2025 Kafka outage: at peak roughly 95% of events rejected over a 38-minute window, caused by a logical error instantiating a new Kafka producer per API request (official report, August 2025). The producer-proliferation rate figure (≈4.2 million extra producers per hour) is from the secondary account in *InfoQ*, “PagerDuty's Kafka Outage Silences Alerts for Thousands of Companies,” 16 September 2025.
5. CodeRabbit. “State of AI vs Human Code Generation” report: analysis of 470 real-world open-source pull requests (320 labeled AI-co-authored, 150 human-only), finding AI-co-authored PRs average 10.83 issues vs 6.45 for human-only (≈1.7×), with logic/correctness issues 75% more common. December 2025. *Vendor-reported* study; the paper cites it as industry data, not independent verification. Business Wire press release, 17 December 2025.
6. Anthropic. “Claude Code Security,” limited research preview announced 20 February 2026; subsequently offered as Claude Security in public beta. An LLM-based codebase scanner that traces data flows, runs a multi-stage adversarial validation pass over its own findings, and proposes patches — with every patch requiring human review and approval. Product announcement and documentation, claude.com/product/claude-security.
7. European Union. Regulation (EU) 2024/2847 (the Cyber Resilience Act), Official Journal of the European Union, 20 November 2024. Annex I, Part II, point (1) mandates a machine-readable software bill of materials covering at least top-level dependencies; Article 14 reporting obligations apply from 11 September 2026; the full set of essential requirements and conformity assessment applies from 11 December 2027.
8. JKI. “J-Crawler” static code analyzer (part of the JKI Security Suite) and National Instruments “VI Analyzer” — static-analysis tooling for the LabVIEW ecosystem, cited as examples of mature code-level checking. jki.net and ni.com documentation.
9. Standard feedback-control terminology (sensor / controller / actuator, setpoint, adaptation) follows any introductory control-engineering text; the paper borrows the architecture and explicitly does *not* claim plant models or stability proofs (see Section 5 and the Conclusion).
10. Yue Huang, Wenjie Wang, Han Bao, Yuchen Ma, Xiaonan Luo, Yi Nian, Haomin Zhuang, Zheyuan Liu, Yue Zhao, and Xiangliang Zhang. “MemoHarness: Agent Harnesses That Learn from Experience.” arXiv:2607.14159 [cs.AI], July 2026. (The authors note a small held-out split and no significance tests; cited here as evidence that experience-driven harness adaptation is implementable, not as validation of its diagnostic reliability.)
11. Weco AI (Zhengyao Jiang, Yuxiang Wu, et al.). “AIDE²: First Evidence of Recursive Self-Improvement.” Company technical report / blog, July 2026. Bi-level harness-code optimization; the authors self-grade the result as RSI Level 1 and report that Level 2 “ignition” was not achieved.

12. The incident-derived failure classes behind the heuristics (actor-cluster, ingestion-pipeline, consensus, and cellular-architecture postmortems) are summarized from public engineering postmortems and outage reports; the curated knowledgebase mapping each heuristic to its originating incident is maintained separately from this paper (see Section 8).